

WorkTogether — Documentation Technique

Application **desktop** de backoffice pour la gestion d'un datacenter. Pile **Tauri v2 + React 19 + Axum (Rust)** connectée à une base **MySQL** servie par WAMP.

Version applicative : 1.0.0 **Identifiant de bundle :** `fr.worktogether.backoffice` **Public visé par ce document :** développeurs, intégrateurs, administrateurs système, mainteneurs. **Dernière mise à jour :** 29 mai 2026

Sommaire

1. Objet et portée du document
 2. Présentation fonctionnelle
 3. Pile technique
 4. Architecture générale
 5. Arborescence du projet
 6. Backend Rust
 - 6.1 Séquence de démarrage
 - 6.2 Configuration de la base (`config.rs`)
 - 6.3 Modèles de données (`models.rs`)
 - 6.4 Couche d'accès aux données (`repository.rs`)
 - 6.5 Commandes Tauri (`main.rs`)
 - 6.6 Serveur HTTP Axum
 - 6.7 État applicatif en mémoire (`AppState`)
 - 6.8 Gestion des erreurs au démarrage
 - 6.9 Système de journalisation
 - 6.10 Binaire utilitaire `create-admin`
 7. Frontend React
 - 7.1 Point d'entrée et routage
 - 7.2 Authentification côté client
 - 7.3 Services JavaScript
 - 7.4 Pages
 - 7.5 Composants réutilisables
 8. Base de données
 9. Sécurité
 10. Permissions et configuration Tauri
 11. Build, exécution et déploiement
 12. Dépannage technique
 13. Annexes
-

1. Objet et portée du document

Ce document décrit l'**architecture logicielle complète** de l'application WorkTogether, du backend Rust jusqu'au frontend React, en passant par la couche d'accès aux données et la sécurité.

Il a pour objectif de permettre à un développeur :

- de comprendre l'organisation du code et la circulation des données ;
- de reprendre la maintenance ou l'évolution de l'application ;
- de compiler, exécuter et déployer le logiciel ;
- de diagnostiquer les incidents techniques.

Il **ne couvre pas** l'utilisation fonctionnelle de l'interface au quotidien : ce point fait l'objet du document séparé *Documentation Utilisateur*.

 **Avertissement de versionnage.** Ce document décrit le code tel qu'il existe au moment de sa rédaction. Certaines valeurs (URL de base de données, ports, identifiants par défaut) sont susceptibles d'évoluer. En cas de divergence, le code source fait foi.

2. Présentation fonctionnelle

WorkTogether est un **outil de backoffice** destiné à la société fictive *WorkTogether Data Solutions*. Il permet de piloter l'exploitation d'un datacenter et son volet commercial à travers une application de bureau native Windows.

Les grands domaines fonctionnels sont :

Domaine	Description	Rôle requis
Authentification	Connexion sécurisée par email + mot de passe (bcrypt)	—
Tableau de bord	Vue de synthèse adaptée au profil de l'utilisateur connecté	Tous
Offres commerciales	Consultation du catalogue d'offres et tarification	Tous
Tickets	Suivi et clôture des demandes d'assistance	Admin, Technicien
Baies serveur	Création, suppression et suivi d'occupation des baies	Admin
Gestion des offres	CRUD complet du catalogue commercial	Admin
Utilisateurs	Création/suppression des comptes applicatifs	Admin
Journaux	Consultation des fichiers de logs de l'application	Admin
Comptabilité	Statistiques financières, clients, réservations	Admin, Comptable

L'application repose sur **trois rôles métier** principaux — `ROLE_ADMIN` , `ROLE_COMPTABLE` , `ROLE_TECHNICIEN` — et un rôle technique de repli `ROLE_USER` .

Périmètre des données

Un point d'architecture essentiel doit être compris dès le départ : **toutes les données ne sont pas persistées de la même manière.**

- Les **baies** (`bay`), les **tickets** (`ticket`) et les **comptes applicatifs** (`app_user`) sont stockés en base MySQL et **persistent** d'un lancement à l'autre.
- Les **offres**, les **clients** et les **réservations** servis par le serveur HTTP interne sont conservés **en mémoire vive** (structures `AppState`). Ils sont **réinitialisés à chaque démarrage** de l'application avec des données d'amorçage (« seed »).

Cette distinction conditionne le comportement attendu : créer une offre via l'interface ne survit pas au redémarrage, alors qu'ajouter une baie est définitif.

3. Pile technique

Couche	Technologie	Version	Rôle
Shell applicatif	Tauri	2.10.2	Fenêtre native, IPC, bundling .exe
Langage backend	Rust	edition 2021	Logique métier, accès BDD
Runtime asynchrone	Tokio	1.x (<code>full</code>)	Exécution async, tâches de fond
Serveur HTTP interne	Axum	0.8.8	API REST locale <code>127.0.0.1:3000</code>
Middleware HTTP	tower-http	0.6.8 (<code>cors</code>)	Gestion CORS
Accès base de données	SQLx	0.8.6 (<code>mysql</code> , <code>chrono</code> , <code>runtime-tokio-native-tls</code>)	Requêtes SQL typées
Base de données	MySQL (WAMP)	3306	Stockage persistant
Hachage mot de passe	bcrypt	0.15	Vérification / création des hashes
Dialogues natifs	tauri-plugin-dialog + rfd	2.x / 0.15	Boîtes de dialogue Win32
Journalisation	tauri-plugin-log + log	2.8 / 0.4	Fichiers <code>.log</code> avec rotation
Configuration	dotenvy	0.15	Lecture de fichiers <code>.env</code>
Sérialisation	serde + serde_json	1.0	JSON <-> structs Rust
Gestion d'erreurs	anyhow	1.0	Erreurs typées au démarrage
Frontend	React	19.1.1	Interface utilisateur
Bundler frontend	Vite	7.1.7	Dev server + build
Routage frontend	React Router DOM	7.12	Navigation SPA
Hachage côté JS	bcryptjs	3.0.3	Dépendance présente (usage marginal)

Justification des choix

- **Tauri plutôt qu'Electron** : binaire final léger (utilise la WebView2 du système au lieu d'embarquer Chromium), démarrage rapide, accès natif à la machine via des « commandes » Rust.
- **Double transport (IPC + HTTP)** : les opérations sensibles (authentification, comptes, logs) passent par l'IPC Tauri, non exposé sur le réseau ; les données analytiques passent par un petit serveur Axum local consommé par `fetch()`.
- **SQLx** : requêtes SQL explicites, mapping automatique vers des structs via `FromRow`, compatibilité MySQL native.

4. Architecture générale

L'application combine **deux canaux de communication** entre le frontend (WebView) et le backend (processus Rust) :



Règles de circulation des données :

- Les actions **sensibles** (auth, comptes, journaux, baies, tickets) utilisent les **commandes Tauri** via `invoke()`. Ce canal n'est accessible que depuis l'application native.
- Les données **analytiques et catalogue** (offres, clients, réservations, statistiques) sont obtenues par **appels HTTP** vers le serveur Axum local via `fetch()`.
- Le serveur Axum partage le **même Repo** que les commandes Tauri pour les données persistées (baies, tickets, stats), mais conserve à part les données volatiles (offres, clients, réservations).

5. Arborescence du projet

```
my-tauri-app/
├── .env # Variable VITE_BACKOFFICE_API_BASE_URL
├── .env.example # Modèle de configuration
├── index.html # Hôte HTML de l'application Vite
├── vite.config.js # Configuration Vite (plugin React)
├── eslint.config.js # Règles ESLint
├── package.json # Dépendances et scripts npm
├── DOCUMENTATION.md # Doc technique historique
├── DOCUMENTATION_TECHNIQUE.md # Ce document
├── DOCUMENTATION_UTILISATEUR.md
├── src/ # — Frontend React —
│   ├── main.jsx # Bootstrap React (providers, router)
│   ├── App.jsx # Définition des routes
│   ├── App.css / index.css # Styles globaux et variables CSS
│   ├── auth/
│   │   ├── AuthContext.jsx # Contexte d'authentification
│   │   ├── Login.jsx # Écran de connexion
│   │   └── RequireRole.jsx # Garde de route par rôle
│   ├── components/
│   │   ├── Menu.jsx # Barre latérale de navigation
│   │   ├── BaieCard.jsx # Carte d'affichage d'une baie
│   │   └── TicketCard.jsx # Carte d'affichage d'un ticket
│   ├── pages/
│   │   ├── Dashboard.jsx
│   │   ├── Offers.jsx
│   │   ├── Tickets.jsx
│   │   ├── AdminBaies.jsx
│   │   ├── AdminOffers.jsx
│   │   ├── AdminUsers.jsx
│   │   ├── AdminLogs.jsx
│   │   ├── ComptableDashboard.jsx
│   │   ├── ComptableClients.jsx
│   │   └── ComptableReservations.jsx
│   └── services/
│       ├── tauri.js # Wrapper invoke + détection runtime
│       ├── authApi.js # Auth + CRUD app_user
│       ├── logger.js # Logging frontend
│       ├── offersApi.js # CRUD offres (HTTP)
│       ├── clientsApi.js # Lecture clients (HTTP)
│       ├── reservationsApi.js # Lecture réservations (HTTP)
│       └── backofficeApi.js # Statistiques comptable (HTTP)
├── src-tauri/ # — Backend Rust —
│   ├── Cargo.toml # Dépendances Rust + binaires
│   ├── tauri.conf.json # Configuration Tauri
│   ├── capabilities/
│   │   └── default.json # Permissions WebView
│   └── src/
│       ├── main.rs # Point d'entrée, commandes, serveur Axum
│       ├── lib.rs # Entrée mobile (non utilisée en desktop)
│       ├── config.rs # Résolution URL BDD + connexion
│       ├── models.rs # Structs SQLx
│       ├── repository.rs # Requêtes SQL
│       └── bin/
│           └── create_admin.rs # CLI de création de compte
```

6. Backend Rust

Le backend est un binaire Rust (`worktogether`) compilé par Tauri. Il contient cinq modules principaux : `main` , `config` , `models` , `repository` , plus un binaire secondaire `create-admin` .

6.1 Séquence de démarrage

La fonction `main()` est annotée `#[tokio::main]` et orchestre la séquence suivante :

```
main()
├── 1. init_db_pool()           ← connexion MySQL (timeout 3 s)
│   ├── succès → pool prêt
│   └── échec → dialogue + « net start wampmysqld64 »
│       ├── 15 tentatives × 2 s (30 s au total)
│       └── échec définitif → dialogue d'erreur + exit(1)
├── 2. Repo::new(pool)         ← instantiation de la couche d'accès
├── 3. repo.run_migration()     ← CREATE TABLE app_user + ALTER bay
│   └── échec → dialogue + exit(1)
├── 4. ensure_initial_admin()   ← crée admin@local.test si app_user vide
├── 5. repo.seed_initial_bays() ← insère B001→B030 (42U) si bay vide
├── 6. AppState::new(repo)      ← amorçage offres/clients/réservations
├── 7. Router Axum + bind 3000  ← serveur HTTP en task::spawn
├── 8. [debug uniquement] attente Vite ← polling TCP :5173 (20 × 500 ms)
└── 9. tauri::Builder           ← plugins log + dialog, setup, run
```

Points notables :

- Le serveur Axum est lancé dans une tâche Tokio détachée (`task::spawn`) : si le bind échoue, l'application continue de fonctionner (les commandes Tauri restent disponibles).
- L'attente de Vite (étape 8) est conditionnée par `#[cfg(debug_assertions)]` : elle n'existe **qu'en mode développement**, pour éviter d'ouvrir la fenêtre avant que le serveur de dev ne réponde.
- Les plugins Tauri (log, dialog) sont enregistrés sur le `Builder` , et le `Repo` est injecté via `.manage(repo)` pour être accessible dans les commandes.

6.2 Configuration de la base (`config.rs`)

Le module `config` résout l'URL de connexion MySQL selon un ordre de priorité strict :

```
1. Variable d'environnement DATABASE_URL
2. Fichier %APPDATA%\WorkTogether\.env (clé DATABASE_URL= ...)
3. Valeur par défaut mysql://root:@localhost:3306/worktogether
```

Fonctions exposées :

Fonction	Signature	Description
<code>get_database_url</code>	<code>fn() → String</code>	Résout l'URL selon la priorité ci-dessus
<code>try_connect</code>	<code>async fn() → Result<MySQLPool></code>	Crée un pool (20 connexions max, timeout d'acquisition 3 s) ; charge aussi le <code>.env</code> local via <code>dotenvy</code>

La constante embarquée est :

```
const DEFAULT_DATABASE_URL: &str = "mysql://root:@localhost:3306/worktogether";
```

Le timeout d'acquisition court (3 s) sert au démarrage : il permet de détecter rapidement l'absence de MySQL pour déclencher la séquence de récupération (`net start wampmysqld64`), au lieu de bloquer l'application.

Changer de base sans recompiler. Créer le fichier `%APPDATA%\WorkTogether\.env` :

```
DATABASE_URL=mysql://utilisateur:motdepasse@hote:3306/ma_base
```

6.3 Modèles de données (`models.rs`)

Toutes les structs dérivent `serde::Serialize` (pour la sérialisation JSON) et, pour celles issues de la base, `sqlx::FromRow` (mapping automatique des colonnes).

```
// Compte applicatif (table app_user)
#[derive(Debug, Serialize, FromRow)]
pub struct AppUser {
    pub id: i32,
    pub email: String,
    pub username: String,
    pub password: String,           // hash bcrypt
    pub roles: String,             // JSON string ex: ["ROLE_ADMIN","ROLE_USER"]
    pub is_active: i8,             // 0/1
}

// Ticket de support (table ticket - schéma Symfony)
#[derive(Debug, Serialize, FromRow)]
pub struct Ticket {
    pub id: i32,
    pub client_id: i32,
    pub title: String,
    pub description: String,
    pub priority: String,           // critical / high / medium / low
    pub status: String,            // open / in_progress / closed
    pub assigned_to: Option<i32>,
}

// Baie serveur (table bay - colonnes capacité ajoutées par migration)
#[derive(Debug, Serialize, FromRow)]
pub struct Bay {
    pub id: i32,
    pub name_bay: String,
    pub units_total: i64,
    pub units_free: i64,
}

// Statistiques comptable (calculé, non issu d'une table)
#[derive(Debug, Serialize)]
pub struct ComptableStats {
    pub total_baies: i32,
    pub total_offres: i32,
    pub total_commandes: i32,
    pub taux_occupation: f64,
}
```

Le champ `roles` est volontairement stocké comme **chaîne JSON** dans une colonne MySQL de type `JSON`. La conversion en `Vec<String>` se fait au niveau de chaque commande qui en a besoin, via `serde_json::from_str`, avec un repli sur `["ROLE_USER"]` en cas d'erreur de parsing.

6.4 Couche d'accès aux données (`repository.rs`)

La struct `Repo` encapsule un `MySQLPool` public et expose des méthodes asynchrones. C'est le **seul point d'accès SQL** de l'application (hors binaire `create-admin`).

Comptes applicatifs

Méthode	SQL exécuté
<code>count_app_users()</code>	<code>SELECT COUNT(*) FROM app_user</code>
<code>get_app_user(email)</code>	<code>SELECT ... FROM app_user WHERE email = ? LIMIT 1</code>
<code>create_app_user(email, username, hash, roles_json)</code>	<code>INSERT INTO app_user (email, username, password, roles) VALUES (?, ?, ?, ?)</code>
<code>delete_app_user(id)</code>	<code>DELETE FROM app_user WHERE id = ?</code>
<code>run_migration()</code>	<code>CREATE TABLE IF NOT EXISTS app_user + ALTER TABLE bay ADD COLUMN ...</code>

Tickets

Méthode	SQL exécuté
<code>get_tickets_open()</code>	<code>SELECT * FROM ticket WHERE status='open' ORDER BY id DESC</code>
<code>close_ticket(id)</code>	<code>UPDATE ticket SET status = 'closed' WHERE id = ?</code>

Baies

Méthode	Détail
<code>seed_initial_bays()</code>	Si <code>bay</code> est vide, insère 30 baies <code>B001</code> → <code>B030</code> à 42 unités. Idempotent : retourne <code>0</code> si au moins une baie existe.
<code>create_bay(name, units_total)</code>	<code>INSERT INTO bay (name_bay, units_total, units_free) VALUES (?, ?, ?)</code> — <code>units_free</code> initialisé à <code>units_total</code> .
<code>delete_bay(id)</code>	<code>DELETE FROM bay WHERE id = ?</code>
<code>get_bays()</code>	<code>SELECT</code> avec <code>LEFT JOIN unit</code> — calcule l'occupation réelle.

La requête `get_bays()` est volontairement robuste : elle combine les colonnes de capacité de la table `bay` avec un décompte réel des `unit` rattachées, en prenant le maximum des deux via `GREATEST` :

```
SELECT b.id, b.name_bay,
       CAST(GREATEST(b.units_total, COUNT(u.id)) AS SIGNED) AS units_total,
       CAST(GREATEST(b.units_free, CAST(COALESCE(SUM(u.is_free), 0) AS SIGNED)) AS SIGNED) AS units_free
FROM bay b
LEFT JOIN unit u ON u.bay_id = b.id
GROUP BY b.id, b.name_bay, b.units_total, b.units_free
ORDER BY b.name_bay
```

Statistiques comptable

`get_comptable_stats()` exécute **quatre requêtes** distinctes :

1. `SELECT COUNT(*) FROM bay` → nombre de baies ;
2. `SELECT COUNT(*) FROM offer` → nombre d'offres en base (table Symfony) ;
3. `SELECT COUNT(*) FROM \ order``` → nombre de commandes ;

4. une sous-requête calculant le **taux d'occupation moyen** des baies (pourcentage d'unités utilisées), retourné en `CHAR` puis converti en `f64` côté Rust (repli sur `0.0`).

6.5 Commandes Tauri (`main.rs`)

Les commandes sont enregistrées dans `invoke_handler` et appelées depuis le frontend via `invoke("nom", { args })`. Les arguments sont désérialisés depuis JSON et nommés en **camelCase** côté JS (Tauri convertit automatiquement vers le `snake_case` Rust).

Commande	Arguments (JS)	Retour	Description
<code>get_tickets</code>	—	<code>Vec<Ticket></code>	Liste des tickets ouverts
<code>close_ticket</code>	<code>{ id }</code>	<code>()</code>	Passe un ticket en <code>closed</code>
<code>get_baies</code>	—	<code>Vec<Bay></code>	Liste des baies avec occupation
<code>add_baie</code>	<code>{ name, unitsTotal }</code>	<code>i32</code> (id)	Crée une baie (≥ 1 unité)
<code>delete_baie</code>	<code>{ id }</code>	<code>()</code>	Supprime une baie
<code>login_db</code>	<code>{ identifiant, password }</code>	<code>AuthUserDto</code>	Authentification bcrypt
<code>list_app_users</code>	—	<code>Vec<AppUserListItem></code>	Liste des comptes applicatifs
<code>create_app_user</code>	<code>{ payload }</code>	<code>i64</code> (id)	Crée un compte (hash bcrypt)
<code>delete_app_user</code>	<code>{ id }</code>	<code>()</code>	Supprime un compte
<code>list_log_files</code>	—	<code>Vec<LogFileInfo></code>	Fichiers <code>.log</code> triés par date
<code>read_log_file</code>	<code>{ filename }</code>	<code>String</code>	Contenu d'un fichier <code>.log</code>
<code>delete_log_file</code>	<code>{ filename }</code>	<code>()</code>	Supprime un fichier <code>.log</code>

Structs de réponse :

```
struct AuthUserDto { id: i32, email: String, username: String, roles: Vec<String> }
struct AppUserListItem { id: i32, email: String, username: String, roles: Vec<String>, is_active: bool }
struct LogFileInfo { name: String, size: u64, modified: String }
```

Détail de `login_db` :

```
async fn login_db(identifiant, password, repo) → Result<AuthUserDto, String> {
    let user = repo.get_app_user(&identifiant).await
        .map_err(|_| "Identifiant ou mot de passe invalide.");
    if user.is_active == 0 { return Err("Ce compte est désactivé."); }
    if !verify_bcrypt_password(&password, &user.password) {
        return Err("Identifiant ou mot de passe invalide.");
    }
    let roles = serde_json::from_str(&user.roles).unwrap_or(vec!["ROLE_USER"]);
    Ok(AuthUserDto { id, email, username, roles })
}
```

L'identifiant attendu est l'**email** (la recherche se fait sur `WHERE email = ?`). Les messages d'erreur sont volontairement génériques pour ne pas révéler si un compte existe.

Validation dans `create_app_user` :

- l'email ne peut pas être vide (après `trim`) ;
- le mot de passe doit faire **au moins 6 caractères** ;
- le mot de passe est haché en bcrypt (`DEFAULT_COST`) avant insertion ;
- les rôles sont sérialisés en JSON ; repli sur `["ROLE_USER"]` si absent.

6.6 Serveur HTTP Axum

Le routeur Axum tourne sur `127.0.0.1:3000` dans une tâche de fond. Il applique `CorsLayer::permissive()` et partage l'état `AppState` (clone léger basé sur des `Arc`).

Route	Méthode	Handler	Source
<code>/api/tickets/open</code>	GET	<code>get_tickets_open</code>	BDD (Repo)
<code>/api/bays</code>	GET	<code>get_bays</code>	BDD (Repo)
<code>/api/bay</code>	POST	<code>create_bay</code>	BDD (Repo)
<code>/api/stats</code>	GET	<code>get_stats</code>	BDD (Repo)
<code>/api/offers</code>	GET	<code>get_offers</code>	Mémoire
<code>/api/offers</code>	POST	<code>create_offer_handler</code>	Mémoire
<code>/api/offers/{id}</code>	PUT	<code>update_offer_handler</code>	Mémoire
<code>/api/offers/{id}</code>	DELETE	<code>delete_offer_handler</code>	Mémoire
<code>/api/clients</code>	GET	<code>get_clients</code>	Mémoire
<code>/api/reservations</code>	GET	<code>get_reservations</code>	Mémoire

Chaque handler retourne un `Json<serde_json::Value>`. En cas d'erreur, la réponse prend la forme `{"error": "message"}` — convention que le frontend interprète comme une erreur même si le code HTTP est 200 (voir `fetchJson` côté client).

Exemple — création d'offre en mémoire :

```
async fn create_offer_handler(State(state), Json(payload)) → Json<Value> {
    let id = state.next_offer_id.fetch_add(1, Ordering::SeqCst); // ID atomique
    let offer = Offer { id, label, units, monthly, monthly_discount };
    state.offers.lock().await.push(offer.clone());
    Json(json!(offer))
}
```

⚠ **Sécurité réseau.** `CorsLayer::permissive()` autorise toutes les origines. Acceptable car le serveur n'écoute que sur la boucle locale (`127.0.0.1`), mais à durcir si le service venait à être exposé.

6.7 État applicatif en mémoire (`AppState`)

`AppState` regroupe les données volatiles et le `Repo` :

```
#[derive(Clone)]
struct AppState {
    repo: Arc<Repo>,
    offers: Arc<Mutex<Vec<Offer>>>,
    clients: Arc<Mutex<Vec<ClientDto>>>,
    reservations: Arc<Mutex<Vec<ReservationDto>>>,
    next_offer_id: Arc<AtomicI32>,
}
```

À l'instanciation (`AppState::new`), les collections sont amorcées :

- **4 offres** : `Base` (1u, 100 €), `Start-up` (10u, 900 €, -10 %), `PME` (21u, 1 680 €, -20 %), `Entreprise` (42u, 2 940 €, -30 %) ;
- **2 clients** : `Acme Corp`, `Globex` ;
- **2 réservations** actives liées à ces clients ;
- `next_offer_id` initialisé à `5` .

Les `Mutex` Tokio garantissent un accès concurrent sûr depuis les handlers asynchrones. **Ces données ne sont jamais écrites en base** : toute modification est perdue au prochain démarrage.

6.8 Gestion des erreurs au démarrage

Deux mécanismes complémentaires coexistent selon le moment :

Avant le démarrage de Tauri → `rfd::MessageDialog` (boîte de dialogue Win32 native, sans WebView).

Utilisé par `init_db_pool()` et la migration, car l' `AppHandle` Tauri n'est pas encore disponible.

```
fn show_dialog(level, title, description) {
    rfd::MessageDialog::new()
        .set_level(level)           // Warning / Error
        .set_title(title)
        .set_description(description)
        .set_buttons(MessageButtons::Ok)
        .show();
}
```

Après le démarrage de Tauri → `tauri-plugin-dialog` (fonction `ask()` côté JS). Utilisé pour les confirmations utilisateur dans l'interface (suppression de baie, d'utilisateur, d'offre, de log, création d'admin).

La séquence de récupération de MySQL est entièrement automatisée :

1. première tentative de connexion ;
2. en cas d'échec : dialogue d'avertissement + commande `net start wampmysql64` ;
3. 15 tentatives espacées de 2 s (soit 30 s) ;
4. échec définitif : dialogue d'erreur détaillé puis `std::process::exit(1)` .

6.9 Système de journalisation

Emplacement des fichiers (Windows) :

```
%APPDATA%\fr.worktogether.backoffice\logs\worktogether.log
```

Configuration du plugin :

```
tauri_plugin_log::Builder::new()
    .target(TargetKind::LogDir { file_name: Some("worktogether") })
    .max_file_size(5_000_000) // 5 Mo par fichier
    .rotation_strategy(RotationStrategy::KeepAll) // conserve tous les fichiers
    .build()
```

Rotation et nettoyage :

- un nouveau fichier est créé lorsque le précédent dépasse 5 Mo ;
- avec `KeepAll`, aucun fichier n'est écrasé automatiquement ;
- au démarrage, `cleanup_old_logs()` supprime tout fichier `.log` dont la date de modification est antérieure à **30 jours**.

Niveaux disponibles : `TRACE` < `DEBUG` < `INFO` < `WARN` < `ERROR`.

Journalisation depuis le frontend (`services/logger.js`) :

```
import { logger } from "../services/logger";

logger.info("message simple");
logger.action("Nom de l'action", { clé: "valeur" }); // → [ACTION] ...
logger.auth("Connexion réussie", { email }); // → [AUTH] ...
logger.warn("attention", { contexte });
logger.error("erreur critique", { détail });
```

Le contexte est formaté en `clé=valeur` JSON et concaténé au message après un `|`.

Actions journalisées automatiquement :

Événement	Niveau	Préfixe
Connexion réussie	INFO	<code>[AUTH]</code>
Déconnexion	INFO	<code>[AUTH]</code>
Création / suppression d'utilisateur	INFO	<code>[ACTION]</code>
Ajout / suppression de baie	INFO	<code>[ACTION]</code>
Création / modification / suppression d'offre	INFO	<code>[ACTION]</code>
Démarrage de l'application	INFO	—

Côté SQLx, les requêtes peuvent apparaître dans les logs. La page « Journaux » du frontend traduit ces requêtes brutes en descriptions lisibles (voir §7.4).

6.10 Binaire utilitaire `create-admin`

Déclaré dans `Cargo.toml` comme binaire secondaire :

```
[[bin]]
name = "create-admin"
path = "src/bin/create_admin.rs"
```

C'est un **outil CLI interactif** permettant de créer un compte `app_user` sans passer par l'interface. Il :

1. lit `DATABASE_URL` depuis l'environnement ou un `.env` ;
2. demande email, username (optionnel), mot de passe (≥ 6 caractères) et rôle ;
3. crée la table `app_user` si nécessaire (même schéma que la migration) ;
4. hache le mot de passe en bcrypt et insère le compte ;
5. gère le cas du doublon d'email (`1062 / Duplicate`).

Usage :

```
cd src-tauri
cargo run --bin create-admin
```

Utile pour réinitialiser l'accès si plus aucun administrateur ne peut se connecter.

7. Frontend React

Le frontend est une **SPA React 19** servie par Vite, rendue dans la WebView Tauri. Il s'appuie sur React Router pour la navigation et un contexte React pour l'authentification.

7.1 Point d'entrée et routage

`main.jsx` monte l'application en enveloppant `<App/>` dans `AuthProvider` puis `BrowserRouter` :

```
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <AuthProvider>  
      <BrowserRouter>  
        <App />  
      </BrowserRouter>  
    </AuthProvider>  
  </StrictMode>  
);
```

`App.jsx` gère deux situations :

- **non authentifié ou route `/`** : seules les routes de connexion sont actives (`Login`), toute autre route redirige vers `/` ;
- **authentifié** : affichage du layout (barre latérale `Menu` + zone de contenu) et des routes protégées.

Table de routage :

Route	Composant	Rôles autorisés
<code>/dashboard</code>	<code>Dashboard</code>	Tous (authentifiés)
<code>/offers</code>	<code>Offers</code>	Tous
<code>/tickets</code>	<code>Tickets</code>	<code>ROLE_ADMIN</code> , <code>ROLE_TECHNICIEN</code>
<code>/admin/baies</code>	<code>AdminBaies</code>	<code>ROLE_ADMIN</code>
<code>/admin/offres</code>	<code>AdminOffers</code>	<code>ROLE_ADMIN</code>
<code>/admin/utilisateurs</code>	<code>AdminUsers</code>	<code>ROLE_ADMIN</code>
<code>/admin/logs</code>	<code>AdminLogs</code>	<code>ROLE_ADMIN</code>
<code>/comptable</code>	<code>ComptableDashboard</code>	<code>ROLE_ADMIN</code> , <code>ROLE_COMPTABLE</code>
<code>/comptable/clients</code>	<code>ComptableClients</code>	<code>ROLE_ADMIN</code> , <code>ROLE_COMPTABLE</code>
<code>/comptable/reservations</code>	<code>ComptableReservations</code>	<code>ROLE_ADMIN</code> , <code>ROLE_COMPTABLE</code>
<code>*</code> (autre)	redirection	→ <code>/dashboard</code>

Chaque route protégée est encapsulée dans `<RequireRole roles={ [ ... ] }>` . Une route sans `roles` (`/dashboard` , `/offers`) n'exige qu'une session authentifiée.

7.2 Authentification côté client

`AuthContext.jsx` expose, via `useAuth()`, l'état et les actions d'authentification :

Valeur	Type	Description
<code>user</code>	objet/null	Utilisateur courant (id, email, username, displayName, roles)
<code>isAuthenticated</code>	booléen	<code>Boolean(user)</code>
<code>loadingAuth</code>	booléen	Restauration de session en cours
<code>authError</code>	string	Dernière erreur d'authentification
<code>login(credentials)</code>	fonction	Connexion (appelle <code>loginWithDatabase</code>)
<code>logout()</code>	fonction	Déconnexion
<code>hasRole(roles)</code>	fonction	Teste l'appartenance à au moins un rôle

Cycle de vie de la session :

1. **Restauration** au montage : `fetchCurrentUser()` relit l'utilisateur depuis `localStorage` (clé `desktop_auth_user`).
2. **Déconnexion automatique à la fermeture** : deux écouteurs sont posés :
 - `beforeunload` (navigateur) : vide le `localStorage` ;
 - `tauri://close-requested` (clic sur la croix) : vide le `localStorage` puis `window.destroy()`.

Ce mécanisme garantit qu'une session ne survit pas à la fermeture de l'application : il faut se reconnecter à chaque lancement.

Garde de route (`RequireRole.jsx`) :

```
if (loadingAuth) return <p>Chargement de la session ... </p>;
if (!user)      return <Navigate to="/" replace />;      // non connecté
if (!hasRole(roles)) return <p>Accès refuse</p>;          // rôle insuffisant
return children;
```

Écran de connexion (`Login.jsx`) : formulaire email + mot de passe, validation locale (champs non vides), affichage des erreurs, état « Connexion en cours... », redirection vers `/dashboard` en cas de succès.

7.3 Services JavaScript

Tous les services sont regroupés dans `src/services/`. Ils isolent la logique de communication du reste de l'interface.

Fichier	Rôle	Transport
<code>tauri.js</code>	Wrapper <code>invoke</code> + détection du runtime Tauri	IPC
<code>authApi.js</code>	Login/logout, CRUD <code>app_user</code> , normalisation des rôles	IPC
<code>logger.js</code>	Journalisation frontend	IPC (plugin-log)
<code>offersApi.js</code>	CRUD offres	HTTP <code>:3000</code>
<code>clientsApi.js</code>	Lecture clients	HTTP <code>:3000</code>
<code>reservationsApi.js</code>	Lecture réservations	HTTP <code>:3000</code>
<code>backofficeApi.js</code>	Statistiques comptable	HTTP <code>:3000</code>

`tauri.js` — détection du runtime :

```
function isTauriRuntime() {
  return typeof window !== "undefined" && Boolean(window.__TAURI_INTERNALS__);
}
export async function invokeCommand(command, args) {
  if (!isTauriRuntime()) throw new Error("Cette fonctionnalité nécessite l'application Tauri...");
  const { invoke } = await import("@tauri-apps/api/core");
  return invoke(command, args);
}
```

Cette détection permet d'afficher un message clair si l'application est ouverte dans un simple navigateur (`npm run dev` sans Tauri) plutôt que de planter.

`authApi.js` — points clés :

- `normaliseRole` met en majuscules, supprime les espaces et remplace par `_` ;
- `extractRoles` / `extractUser` normalisent la réponse du backend ;
- la session est stockée/relue dans `localStorage` ;
- `hasAnyRole(user, expectedRoles)` retourne `true` si la liste attendue est vide (route ouverte à tout authentifié), sinon teste l'intersection.

Services HTTP : tous partagent un helper `fetchJson` identique qui :

- construit l'URL à partir de `VITE_BACKOFFICE_API_BASE_URL` (défaut `http://127.0.0.1:3000`) ;
- pose les en-têtes `Accept` / `Content-Type` ;
- traite `{"error": ...}` comme une erreur même en HTTP 200.

7.4 Pages

Dashboard (`/dashboard`)

Page d'accueil adaptée au rôle. Charge en parallèle, selon les droits :

- statistiques comptable (`getComptableStats`) et réservations si Comptable/Admin ;
- tickets ouverts (`get_tickets`) si Technicien/Admin.

Affiche des **KPI** (tickets ouverts, baies, offres, commandes, taux d'occupation), une liste des **tickets ouverts récents** (4 max), des **réservations récentes** et une grille d'**accès rapides** dépendant du rôle. Le salut (« Bonjour / Bon après-midi / Bonsoir ») dépend de l'heure.

Offres (/offers)

Catalogue **en lecture seule** alimenté par `getOffers()`. Affiche un tableau (offre, unités, prix mensuel, remise, prix annuel) et des statistiques (nombre d'offres, prix mini/maxi). Le prix annuel affiché applique une réduction forfaitaire de **-10 %** ($\text{monthly} \times 12 \times 0.9$).

Tickets (/tickets)

Gestion des tickets via les commandes Tauri `get_tickets` et `close_ticket`. Fonctionnalités :

- statistiques (ouverts, critiques, clôturés, total) ;
- filtres **Ouverts / Clôturés / Tous** ;
- tri par priorité (`critical` < `high` < `medium` < `low`) ;
- clôture d'un ticket avec rechargement automatique.

Administration des baies (/admin/baies)

Liste des baies (`get_baies`) sous forme de cartes (`BaieCard`), statistiques globales (baies, unités totales/libres, taux d'occupation), formulaire d'ajout (`add_baie`) et suppression avec confirmation native (`ask`). Chaque action est journalisée.

Gestion des offres (/admin/offres)

CRUD complet via `offersApi`. Particularités :

- chargement conjoint des offres et des réservations pour marquer les offres « actives » (utilisées par une réservation) ;
- une offre utilisée par une réservation **ne peut pas être supprimée** (bouton désactivé) ;
- mode édition avec pré-remplissage du formulaire ;
- aperçu du prix après remise en temps réel.

Gestion des utilisateurs (/admin/utilisateurs)

Liste des comptes (`list_app_users`), création (`create_app_user`) et suppression (`delete_app_user`). Particularités :

- sélection du rôle parmi Administrateur / Comptable / Technicien ;
- validation du mot de passe (≥ 6 caractères) avec indicateur visuel ;
- **confirmation renforcée** lors de la création d'un compte administrateur ;
- avertissement spécifique à la suppression d'un compte administrateur ;
- bouton « afficher/masquer » le mot de passe.

Journaux (/admin/logs)

Visionneuse de logs basée sur `list_log_files`, `read_log_file`, `delete_log_file`. Fonctionnalités avancées :

- sélecteur de fichier (nom, taille, date) ;
- compteurs par niveau (INFO / WARN / ERROR) ;
- filtres par niveau et **recherche plein texte** ;

- **humanisation des requêtes SQL** : une table de règles (`SQL_RULES`) traduit les requêtes brutes en descriptions lisibles (ex. `SELECT * FROM ticket WHERE status='open'` → « Lecture des tickets ouverts ») ;
- défilement automatique optionnel ;
- suppression du fichier sélectionné avec confirmation.

Espace comptable

- **Tableau de bord** (`/comptable`) : KPI financiers (revenu mensuel actif, commandes, clients, offres, baies, taux d'occupation), réservations récentes, **top clients par revenu** (barres de progression), répartition actives/inactives.
- **Clients** (`/comptable/clients`) : répertoire avec recherche (nom, email, société), avatars à initiales, lien `mailto` .
- **Réservations** (`/comptable/reservations`) : tableau complet avec filtres par statut (`active` , `pending` , `cancelled` , `expired`), statistiques (total, actives, chiffre d'affaires cumulé).

7.5 Composants réutilisables

Composant	Rôle
<code>Menu.jsx</code>	Barre latérale : logo, sections de navigation conditionnées par le rôle , pied avec avatar/rôle et bouton de déconnexion
<code>BaieCard.jsx</code>	Carte d'une baie : nom, ID, badge de pourcentage, barre de progression colorée (vert < 60 %, orange < 90 %, rouge ≥ 90 %), totaux
<code>TicketCard.jsx</code>	Carte d'un ticket : pastille de priorité, badges priorité/statut, description, méta (ticket/client/assigné), bouton « Clôturer »

Le `Menu` n'affiche les sections **Support**, **Comptabilité** et **Administration** que si l'utilisateur possède le rôle correspondant, via `hasRole( ... )` .

8. Base de données

Connexion par défaut : `mysql://root:@localhost:3306/worktogether`

Tables utilisées

Table	Origine	Usage par l'application
<code>app_user</code>	Tauri (auto-migrée)	Lecture/écriture complète (auth + comptes)
<code>bay</code>	Symfony	Lecture/écriture + colonnes <code>units_total</code> , <code>units_free</code> ajoutées
<code>unit</code>	Symfony	Lecture seule (calcul d'occupation)
<code>ticket</code>	Symfony	Lecture + mise à jour du statut
<code>offer</code>	Symfony	Lecture seule (comptage stats)
<code>order</code>	Symfony	Lecture seule (comptage stats)
<code>user</code>	Symfony	Non utilisée par l'app (auth via <code>app_user</code>)

Schéma de `app_user`

```
CREATE TABLE IF NOT EXISTS `app_user` (  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `email` VARCHAR(180) NOT NULL,  
  `username` VARCHAR(100) NULL,  
  `password` VARCHAR(255) NOT NULL,  
  `roles` JSON NOT NULL DEFAULT (JSON_ARRAY('ROLE_USER')),  
  `is_active` TINYINT(1) NOT NULL DEFAULT 1,  
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uniq_app_user_email` (`email`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Migration automatique (`run_migration`)

Exécutée au démarrage, **idempotente** :

```
CREATE TABLE IF NOT EXISTS app_user ( ... ) -- crée si absente  
ALTER TABLE bay ADD COLUMN units_total ... -- ignorée si déjà présente  
ALTER TABLE bay ADD COLUMN units_free ... -- ignorée si déjà présente
```

Les `ALTER TABLE` sont volontairement exécutés sans propagation d'erreur (`let _ = ...`) : si les colonnes existent déjà, l'échec est ignoré.

Amorçage des baies (`seed_initial_bays`)

Si la table `bay` est vide, l'application insère **30 baies** nommées `B001` à `B030` , chacune avec 42 unités (`units_total = units_free = 42`), conformément au cahier des charges. Opération idempotente.

Compte administrateur initial

Au tout premier lancement (table `app_user` vide), un compte est créé automatiquement :

```
email      : admin@local.test
password   : admin123!
roles      : ["ROLE_ADMIN", "ROLE_USER"]
```

 **À changer immédiatement** après la première connexion (créer un nouvel admin puis supprimer celui-ci, ou utiliser `create-admin`).

9. Sécurité

Mots de passe

- Hachage **bcrypt** (`DEFAULT_COST`) à la création.
- Vérification via `verify_bcrypt_password` qui gère la **compatibilité PHP/Symfony** : un hash `$2y$` (PHP) est normalisé en `$2b$` (Rust) avant comparaison.

```
fn verify_bcrypt_password(password, hash) → bool {
    if verify(password, hash).unwrap_or(false) { return true; }
    if hash.starts_with("$2y$") {
        let normalized = hash.replacen("$2y$", "$2b$", 1);
        return verify(password, &normalized).unwrap_or(false);
    }
    false
}
```

Accès aux fichiers de logs

Les commandes `read_log_file` et `delete_log_file` se protègent contre la **traversée de répertoire** :

- rejet de tout nom contenant `/`, `\` ou `..` ;
- vérification que l'extension est `.log` ;
- chemin **toujours** reconstruit depuis `app_log_dir()`, jamais depuis l'entrée brute.

Contrôle d'accès

- Côté backend, les commandes ne re-vérifient pas le rôle : la protection repose sur le frontend (`RequireRole`) et le fait que l'IPC n'est pas exposé hors de l'application.
- Côté frontend, chaque route sensible est gardée par rôle.

Surface réseau

- Le serveur Axum n'écoute que sur `127.0.0.1:3000` (boucle locale).
- CORS permissif — sans impact tant que le service reste local.

Recommandations de durcissement

1. Forcer le changement du mot de passe admin par défaut au premier login.
 2. Ajouter une vérification de rôle côté commandes Tauri (défense en profondeur).
 3. Restreindre le CORS si le port 3000 venait à être exposé.
 4. Envisager le chiffrement au repos de la base si des données sensibles y sont ajoutées.
-

10. Permissions et configuration Tauri

Capacités (`capabilities/default.json`)

```
{
  "identifiant": "default",
  "windows": ["main"],
  "permissions": [
    "core:default",
    "core:window:allow-destroy",
    "core:event:allow-listen",
    "core:event:allow-unlisten",
    "dialog:allow-ask",
    "dialog:allow-confirm",
    "dialog:allow-message",
    "log:allow-log"
  ]
}
```

Pour ajouter une fonctionnalité native (accès fichiers étendu, notifications...), il faut **à la fois** ajouter la permission ici **et** enregistrer le plugin correspondant dans `main.rs`.

Configuration applicative (`tauri.conf.json`)

Clé	Valeur
<code>productName</code>	<code>WorkTogether</code>
<code>version</code>	<code>1.0.0</code>
<code>identifiant</code>	<code>fr.worktogether.backoffice</code>
<code>build.frontendDist</code>	<code>../dist</code>
<code>build.devUrl</code>	<code>http://localhost:5173</code>
<code>build.beforeDevCommand</code>	<code>npm run dev</code>
<code>build.beforeBuildCommand</code>	<code>npm run build</code>
Fenêtre	titre <code>WorkTogether - Backoffice</code> , 1280×800, redimensionnable
<code>security.csp</code>	<code>null</code> (aucune politique CSP)
<code>bundle.targets</code>	<code>all</code>

11. Build, exécution et déploiement

Prérequis

- **Node.js 20+** et npm
- **Rust + Tauri CLI** (`@tauri-apps/cli` est en devDependency)
- **WAMP** (MySQL sur le port 3306) avec la base `worktogether`
- **WebView2** (pré-installé sur Windows 11)

Commandes courantes

```
# Installation des dépendances
cd my-tauri-app
npm install

# Développement (lance Vite + Tauri)
npm run tauri dev

# Build production (génère l'exécutable autonome)
npm run tauri build
# Résultat : src-tauri/target/release/WorkTogether.exe
# Installeur: src-tauri/target/release/bundle/

# Build frontend seul
npm run build

# Vérification Rust rapide (sans binaire final)
cd src-tauri ; cargo check

# Rebuild Rust propre (long)
cd src-tauri ; cargo clean ; cd .. ; npm run tauri build

# Outil de création de compte
cd src-tauri ; cargo run --bin create-admin

# Suivre les logs en direct (PowerShell)
Get-Content "$env:APPDATA\fr.worktogether.backoffice\logs\worktogether.log" -Wait
```

Prérequis d'exécution du `.exe`

1. WAMP démarré (MySQL :3306) — ou le service `wampmysqld64` doit pouvoir être lancé par l'application (droits administrateur requis).
2. Base `worktogether` existante avec le schéma Symfony déployé.
3. WebView2 présent sur le poste.

Note sur le mode développement seul

En lançant uniquement `npm run dev` (sans Tauri), l'interface s'ouvre dans un navigateur mais **les commandes natives ne fonctionnent pas** : `isTauriRuntime()` renvoie `false` et les appels IPC lèvent une erreur explicite. Les pages basées sur le serveur HTTP (offres, clients...) ne fonctionnent que si le backend Rust tourne par ailleurs.

12. Dépannage technique

Symptôme	Cause probable	Résolution
Dialogue « MySQL n'est pas accessible » au lancement	WAMP arrêté ou service introuvable	Démarrer WAMP ; lancer l'app en administrateur
« Impossible de démarrer MySQL après 30 secondes » puis fermeture	Droits insuffisants, WAMP non installé, base absente	Vérifier l'installation WAMP et l'existence de la base <code>worktogether</code>
Échec de migration au démarrage	Permissions SQL insuffisantes, base inaccessible	Vérifier les droits de l'utilisateur MySQL
Les offres/clients/réservations sont vides ou « par défaut »	Données en mémoire réinitialisées	Comportement normal : ces données ne persistent pas
« Cette fonctionnalité nécessite l'application Tauri »	App ouverte dans un navigateur classique	Lancer via <code>npm run tauri dev</code> ou le <code>.exe</code>
Erreur API (offres/clients) dans l'interface	Serveur Axum non démarré (port 3000 occupé)	Vérifier les logs ; libérer le port 3000
Connexion refusée malgré le bon mot de passe	Compte désactivé (<code>is_active = 0</code>) ou email erroné	Vérifier l'état du compte en base
Page « Accès refuse »	Rôle insuffisant pour la route	Vérifier les rôles du compte

Où regarder en priorité

1. Les **fichiers de logs** (`%APPDATA%\fr.worktogether.backoffice\logs\`) ou la page **Journaux** de l'application.
2. La **console de dev** Vite/WebView en mode `tauri dev`.
3. La **connexion MySQL** (`mysql -u root` sur le port 3306).

13. Annexes

A. Contrats d'API HTTP (exemples JSON)

GET /api/offers

```
[
  { "id": 1, "label": "Base", "units": 1, "monthly": 100, "monthlyDiscount": 0 }
]
```

POST /api/offers — corps de requête :

```
{ "label": "Pro", "units": 5, "monthly": 450, "monthlyDiscount": 5 }
```

GET /api/stats

```
{ "total_baies": 30, "total_offres": 4, "total_commandes": 12, "taux_occupation": 37.5 }
```

GET /api/reservations

```
[
  {
    "id": 1, "client_id": 1, "client_name": "Acme Corp",
    "offer_id": 2, "offer_name": "Start-up", "amount": 900,
    "start_date": "2026-01-01T00:00:00Z", "end_date": "2026-01-31T23:59:59Z",
    "status": "active"
  }
]
```

B. Réponse de `login_db` (IPC)

```
{ "id": 1, "email": "admin@local.test", "username": "admin", "roles": ["ROLE_ADMIN", "ROLE_USER"] }
```

C. Glossaire

Terme	Définition
Baie	Armoire de datacenter contenant des unités (U) d'équipement.
Unité (U)	Emplacement standard d'une baie ; une baie en compte 42 par défaut.
IPC	<i>Inter-Process Communication</i> — canal entre la WebView et le backend Rust.
Commande Tauri	Fonction Rust exposée au frontend via <code>invoke()</code> .
AppState	Données volatiles en mémoire (offres, clients, réservations).
Seed	Données d'amorçage insérées au démarrage.
Migration	Évolution idempotente du schéma exécutée au démarrage.
bcrypt	Algorithme de hachage de mots de passe.
WebView2	Moteur de rendu web fourni par Windows, utilisé par Tauri.

D. Rôles applicatifs

Rôle	Label	Périmètre
<code>ROLE_ADMIN</code>	Administrateur	Accès total
<code>ROLE_COMPTABLE</code>	Comptable	Comptabilité, clients, réservations, offres (lecture)
<code>ROLE_TECHNICIEN</code>	Technicien	Tickets
<code>ROLE_USER</code>	Utilisateur	Rôle technique de repli (dashboard, offres)